

CrackMe №3

Task

Author	koboldschwaengel
Language	C\C++
Platform	Windows
Arch	x86-64
Difficulty	3.0

Description

This is my first crackme. I would say it is between difficulty 3 and 4. Have fun!

Task source

Решение

Обозревая строки бинарника, видим те, по которым, возможно, будет можно выйти на валидацию пароля:

```
.rdata:32f0 "Enter the Password:"  
.rdata:3308 "Correct!"  
.rdata:3318 "Wrong password"
```

По референсам из .rdata переходим в main , где видим запрос пароля. Пароль записывается в массив Buffer:

```
char Buffer[16];  
..  
Stream = _acrt_iob_func(0);  
if (fgets(Buffer,11, Stream))...  
...  
v10=((unsigned __int8 (__fastcall*)(char*, __int64))lpAddress)(Buffer, n0xB_1);
```

А затем есть очень интересный switch-case

```
if(IsDebuggerPresent()) LOBYTE(v10) =0;  
else v10=(lpAddress)(Buffer, n0xB_1)  
n23 = 23;  
v12 = 0;  
while(2)  
{  
    switch(n23)  
    {  
        case 23:  
            n23=153;  
            v12 = ((unsigned __int8)v10^0x55u)+3;  
            if( (((unsigned __int8)v10^0x55u)+3)&1)!=0)  
                n23=66;  
            continue;
```

```

    case 33:
        v12=(unsigned int)v12^0x1337;
        n23=23;
        if( (v12&8)!=0)
            n23=66;
        continue;

    case 66:
        v12 = (4*n23)^(unsigned int)v12;
        n23=33;
        if( (_DWORD)v12 == 3*((int)v12/3))
            n23=102
        continue;

    case 102:
        n23=222;
        if( ((unsigned int)v12^(unsigned __int8)v10)!=0x15E)
            n23=173;
        continue;

    case 153:
        v12=(unsigned int)(n23+v12);
        n23=102;
        if( (v12&4)!=0)
            n23=33;
        continue;

    case 173:
        Wrong_password="Wrong password";
        break;

    case 222:
        Wrong_password="Correct!";
        break;
}
}

```

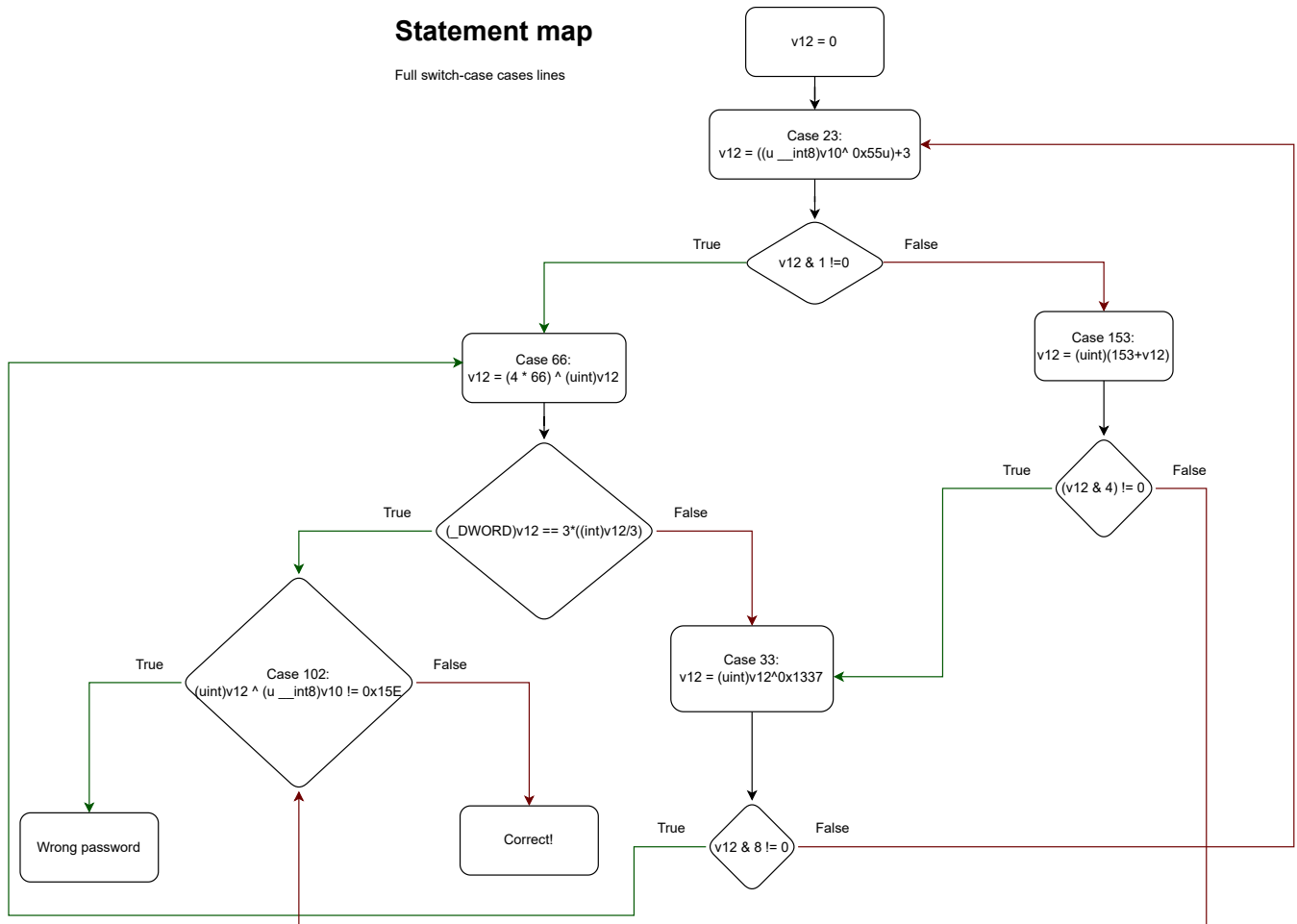
Решающим является case 102

Если $(\text{unsigned int})v12 \wedge (\text{unsigned int})v10 == 0x15E$, то пароль верный.

Осталось составить цепочку состояний $v12$ в соответствии с серией проверок switch-case:

Statement map

Full switch-case cases lines



Но есть одна интересная вещь:

```
v10 = ((unsigned __int8 (__fastcall *)(char *, __int64))lpAddress)(Buffer,n0xB_1);
```

Решил подробнее изучить эту JIT-функцию в процессе работы программы:

```
call rdi
```

В регистрах на момент вызова:

```
RAX : 0000000000000000
RBX : 0000000000000007
RCX : 000000F46A4FFDF8    "1234567"
RDX : 0000000000000007
RBP : 000000F46A4FFDB9
RSP : 000000F46A4FFD50
RSI : 000002988E190071
RDI : 000002988E190000
R8  : 000000F46A4FFCF8
R9  : 0000000000000040    '@'
R10 : 0000000000000000
R11 : 0000000000000246    L'Ѓ'
R12 : 0000000000000000
R13 : 0000000000000000
R14 : 0000000000000000
R15 : 0000000000000000
RIP : 00007FF66F4F1292    crackme.00007FF66F4F1292
```

```
RFLAGS : 0000000000000246      L'ǻ'
ZF : 1
OF : 0
CF : 0
PF : 1
SF : 0
TF : 0      L'Ä'
AF : 0
DF : 0
IF : 1
```

Посмотрим, что лежит по адресу в регистре rdi:

```
000002988E190000 48 83 EC 18 4C 8B D1 48 83 FA 07 75 5D 33 C0 C7 H.i.L.ÑH.ú.u]3ÀÇ
000002988E190010 04 24 70 61 73 73 48 8D 0C 24 66 C7 44 24 04 31 $.passH..$fÇD$.1
000002988E190020 32 4C 2B D1 C6 44 24 06 33 0F 1F 80 00 00 00 00 2L+ÑÆD$.3.....
000002988E190030 4C 8D 04 24 4C 03 C0 47 0F B6 0C 02 45 84 C9 74 L..$.L.ÀG.¶..E.Ét
000002988E190040 29 0F B6 C8 BA AD DE 00 00 80 E1 01 C0 E1 03 66 ).¶È°.D...á.Àá.f
000002988E190050 D3 EA 41 32 D1 41 3A 10 75 10 48 FF C0 48 83 F8 ÓêA2ÑA:..u.HÿÀH.ø
000002988E190060 07 72 CD B0 01 48 83 C4 18 C3 32 C0 48 83 C4 18 .rÍ°.H.Ä.Ã2ÄH.Ä.
000002988E190070 C3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 Ä.....
```

Или, если дизассемблировать:

```
000002988E190000 | 48:83EC 18 | sub rsp,18 |
000002988E190004 | 4C:8BD1 | mov r10,rcx | rcx:"1234567"
000002988E190007 | 48:83FA 07 | cmp rdx,7 |
000002988E19000B | 75 5D | jne 2988E19006A |
000002988E19000D | 33C0 | xor eax,eax |
000002988E19000F | C70424 70617373 | mov dword ptr ss:[rsp],73736170 |
000002988E190016 | 48:8D0C24 | lea rcx,qword ptr ss:[rsp] |
000002988E19001A | 66:C74424 04 3132 | mov word ptr ss:[rsp+4],3231 |
000002988E190021 | 4C:2BD1 | sub r10,rcx | rcx:"1234567"
000002988E190024 | C64424 06 33 | mov byte ptr ss:[rsp+6],33 | 33:'3'
000002988E190029 | 0F1F80 00000000 | nop dword ptr ds:[rax],eax |
000002988E190030 | 4C:8D0424 | lea r8,qword ptr ss:[rsp] |
000002988E190034 | 4C:03C0 | add r8,rax |
000002988E190037 | 47:0FB60C02 | movzx r9d,byte ptr ds:[r10+r8] |
000002988E19003C | 45:84C9 | test r9b,r9b |
000002988E19003F | 74 29 | je 2988E19006A |
000002988E190041 | 0FB6C8 | movzx ecx,al |
000002988E190044 | BA ADDE0000 | mov edx,DEAD |
000002988E190049 | 80E1 01 | and cl,1 |
000002988E19004C | C0E1 03 | shl cl,3 |
000002988E19004F | 66:D3EA | shr dx,cl |
000002988E190052 | 41:32D1 | xor dl,r9b |
000002988E190055 | 41:3A10 | cmp dl,byte ptr ds:[r8] |
000002988E190058 | 75 10 | jne 2988E19006A |
000002988E19005A | 48:FFC0 | inc rax |
000002988E19005D | 48:83F8 07 | cmp rax,7 |
000002988E190061 | 72 CD | jb 2988E190030 |
000002988E190063 | B0 01 | mov al,1 |
000002988E190065 | 48:83C4 18 | add rsp,18 |
000002988E190069 | C3 | ret |
000002988E19006A | 32C0 | xor al,al |
000002988E19006C | 48:83C4 18 | add rsp,18 |
000002988E190070 | C3 | ret |
```

И вот что мы получаем в регистрах на момент возврата из этой функции:

```
RAX : 0000000000000000
RBX : 0000000000000007
RCX : 0000000000000000
RDX : 000000000000DE9C
RBP : 000000F46A4FFDB9
RSP : 000000F46A4FFD48
RSI : 000002988E190071
RDI : 000002988E190000
R8  : 000000F46A4FFD30    "pass123"
R9  : 0000000000000031    '1'
R10 : 00000000000000C8    'È'
R11 : 0000000000000246    L'ž'
R12 : 0000000000000000
R13 : 0000000000000000
R14 : 0000000000000000
R15 : 0000000000000000
RIP : 000002988E190070
RFLAGS : 0000000000000206    L'Ê'
ZF : 0
OF : 0
CF : 0
PF : 1
SF : 0
TF : 0    L'Ä'
AF : 0
DF : 0
IF : 1
```

Пароль pass123 ожидаемо не проходит. Ловушка для новичка.

Посмотрим внимательнее:

```
mov edx, DEAD
and cl, 1
shl cl, 3
shr xd, cl
xor dl, r9b
cmp dl, byte ptr ds:[r8] // r8 = "pass123"/70 61 73 73 31 32 33
```

Посмотрим первую итерацию:

```
input: 1234567
r9 = 1
rdx=dead
...
cmp dl, byte ptr ds:[r8]
dl = 9C, r8 = "pass123"/70 61 73 73 31 32 33
r9 = 31
31 ^ AD = 9C
```

Получается, хог происходит либо с 0xDE. либо с 0xAD в зависимости от позиции элемента.

Проверим одну гипотезу:

```
pass123 ^ AD = DD CC DE DE 9C 9F 9E
```

```
pass123 ^ DE = AE BF AD AD EF EC ED
```

И напишем скрипт, который выполняет хог в зависимости от позиции

```
target = b"pass123"
password=bytes(c^(0xad if i%2==0 else 0xde) for i, c in enumerate(target))
print(password.hex())
```

OUT: ddbfdead9cec9e

target	70	61	73	73	31	31	33
pass123 ^ 0xDE	AE	BF	AD	AD	EF	EC	ED
pass123 ^ 0xAD	DD	CC	DE	DE	9C	9F	9E
script output	DD	BF	DE	AD	9E	EC	9E

Валидный пароль: \0xDD \0xBF \0xDE \0xAD \0x9E \0xEC \0x9E